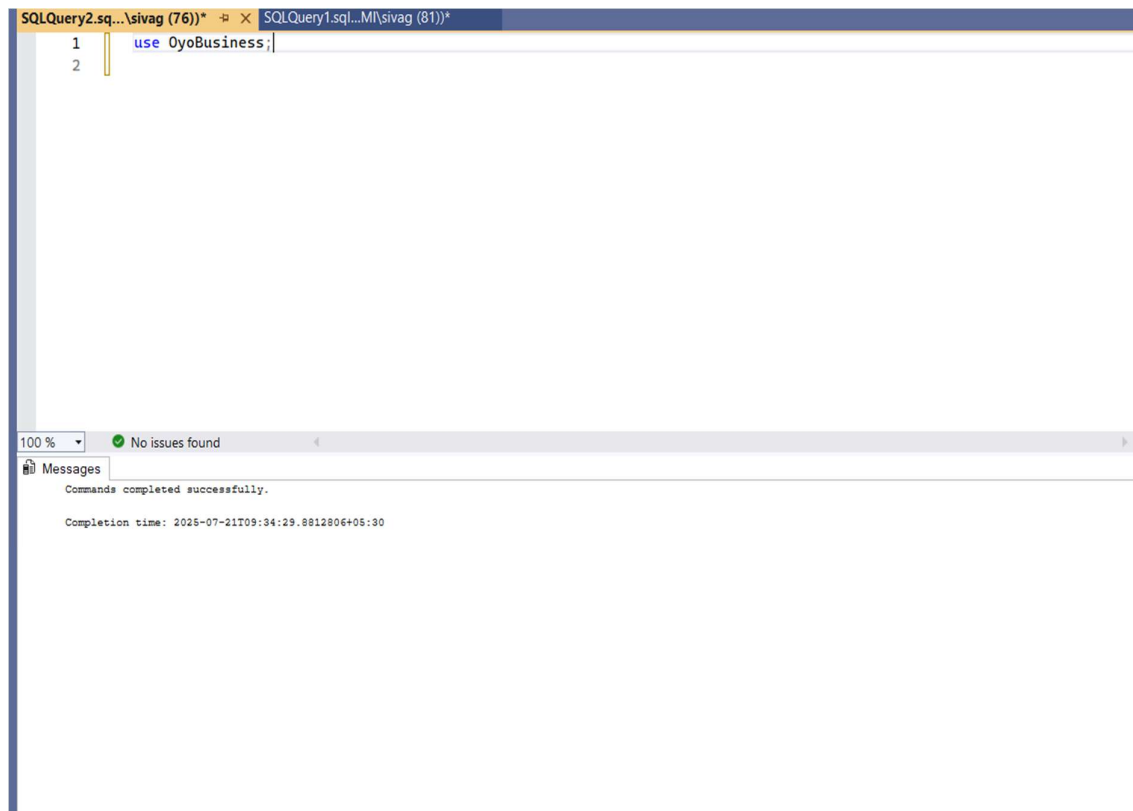


Documentation – OyoBusiness SQL Queries

Step 1: Using the Database

Database Used: OyoBusiness

- This project is based on a database named OyoBusiness, which contains real-time hotel booking data.
- It includes two tables:
 - bookings – contains information like booking dates, customer ID, hotel ID, number of rooms, amount, discount, etc.
 - hotels – maps hotel IDs to their respective city names.



The screenshot shows a SQL IDE interface with two tabs at the top: 'SQLQuery2.sql...\\sivag (76))' and 'SQLQuery1.sql...MI\\sivag (81))'. The active tab contains the SQL command 'use OyoBusiness;' on line 1. Below the editor, a status bar indicates '100 %' zoom and 'No issues found'. A 'Messages' panel at the bottom displays the message 'Commands completed successfully.' and the completion time '2025-07-21T09:34:29.8812806+05:30'.

```
SQLQuery2.sql...\\sivag (76))  SQLQuery1.sql...MI\\sivag (81))
1  use OyoBusiness;
2
```

100 % No issues found

Messages

Commands completed successfully.

Completion time: 2025-07-21T09:34:29.8812806+05:30

Bookings Table

SQLQuery2.sql...sivag (76))*SQLQuery1.sql...M\sivag (81))*

1

2

3

4

use OyoBusiness;

select * from bookings;

100 %

No issues found

Results

Messages

	booking_id	customer_id	status	check_in	check_out	no_of_rooms	hotel_id	amount	discount	date_of_booking
1	170	9197	Cancelled	2022-01-14 00:00:00.000	2022-01-15 00:00:00.000	1	757	3137	796	2022-01-01 00:00:00.000
2	171	9197	Cancelled	2022-01-16 00:00:00.000	2022-01-17 00:00:00.000	1	757	3137	663	2022-01-01 00:00:00.000
3	583	11659	Stayed	2022-01-01 00:00:00.000	2022-01-02 00:00:00.000	2	16	6237	1320	2022-01-01 00:00:00.000
4	642	14499	Cancelled	2022-01-16 00:00:00.000	2022-01-18 00:00:00.000	10	346	59254	12535	2022-01-01 00:00:00.000
5	569	15037	No Show	2022-01-02 00:00:00.000	2022-01-03 00:00:00.000	1	207	1884	398	2022-01-01 00:00:00.000
6	882	22257	Cancelled	2022-01-01 00:00:00.000	2022-01-02 00:00:00.000	1	800	3444	875	2022-01-01 00:00:00.000
7	419	29124	Cancelled	2022-01-02 00:00:00.000	2022-01-03 00:00:00.000	1	380	2144	453	2022-01-01 00:00:00.000
8	436	33021	No Show	2022-01-01 00:00:00.000	2022-01-02 00:00:00.000	1	62	2252	476	2022-01-01 00:00:00.000
9	889	65208	Stayed	2022-01-01 00:00:00.000	2022-01-02 00:00:00.000	1	117	5589	1419	2022-01-01 00:00:00.000
10	17	66762	Stayed	2022-01-07 00:00:00.000	2022-01-08 00:00:00.000	1	109	3509	743	2022-01-01 00:00:00.000
11	153	77179	Stayed	2022-01-02 00:00:00.000	2022-01-06 00:00:00.000	1	106	11887	2515	2022-01-01 00:00:00.000
12	545	81709	Cancelled	2022-01-02 00:00:00.000	2022-01-03 00:00:00.000	1	449	2192	464	2022-01-01 00:00:00.000
13	416	86098	Stayed	2022-01-01 00:00:00.000	2022-01-02 00:00:00.000	1	579	2599	550	2022-01-01 00:00:00.000
14	641	86311	Stayed	2022-01-01 00:00:00.000	2022-01-02 00:00:00.000	1	346	3199	677	2022-01-01 00:00:00.000
15	491	102430	Cancelled	2022-01-02 00:00:00.000	2022-01-03 00:00:00.000	1	3	2469	523	2022-01-01 00:00:00.000
16	149	112913	Stayed	2022-01-29 00:00:00.000	2022-02-02 00:00:00.000	1	409	26255	5554	2022-01-01 00:00:00.000
17	799	118118	Stayed	2022-01-01 00:00:00.000	2022-01-02 00:00:00.000	1	689	4224	893	2022-01-01 00:00:00.000

Query executed successfully.

SIVAKAMI\SQLEXPRESS01 (16.0...SIVAKAMI\sivag (76)OyoB

Hotels Table

SQLQuery2.sql...sivag (76))*SQLQuery1.sql...M\sivag (81))*

1

2

3

4

use OyoBusiness;

select * from bookings;

select * from hotels;

100 %

No issues found

Results

Messages

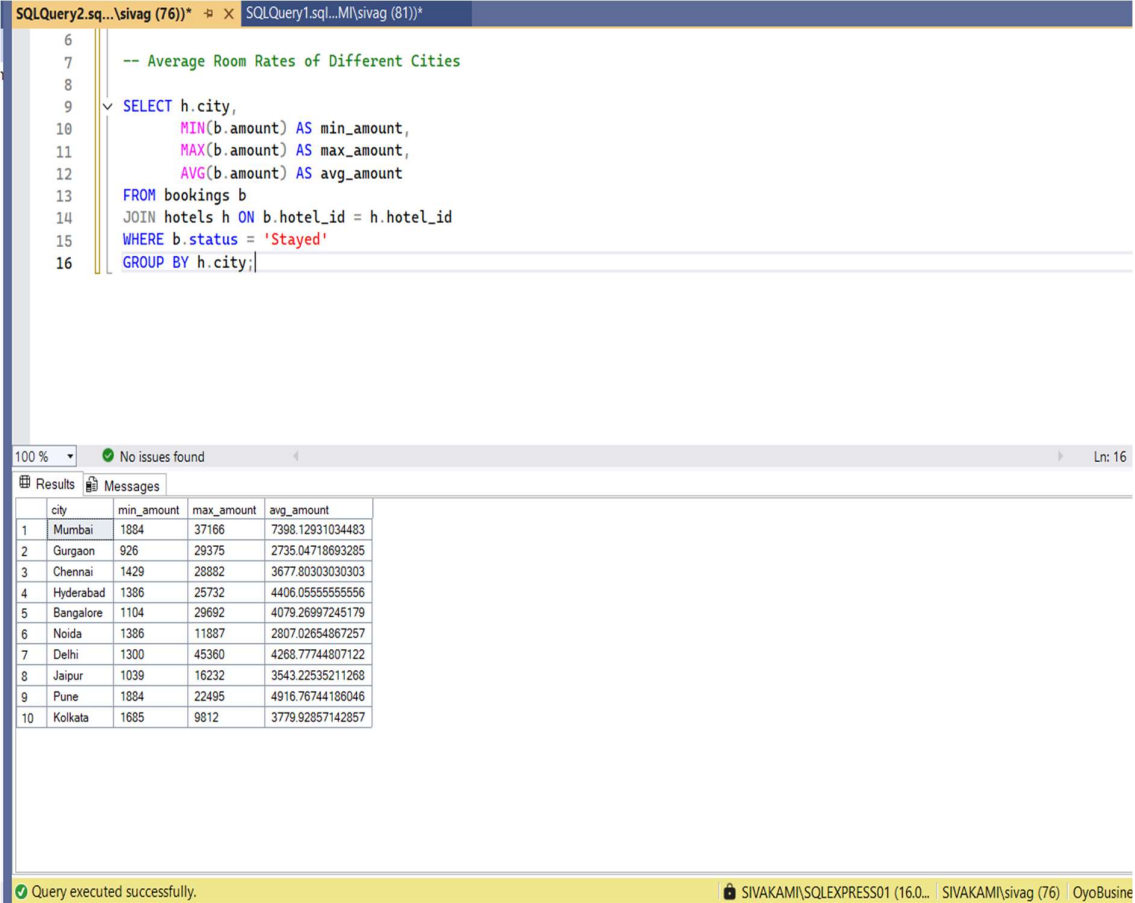
	hotel_id	city
1	3	Gurgaon
2	13	Gurgaon
3	16	Gurgaon
4	21	Gurgaon
5	25	Delhi
6	29	Gurgaon
7	31	Delhi
8	33	Gurgaon
9	37	Gurgaon
10	44	Noida
11	45	Delhi
12	48	Delhi
13	49	Delhi
14	50	Gurgaon
15	56	Bangalore
16	58	Gurgaon
17	59	Noida

Query executed successfully.

SIVAKAMI\SQLEXPRESS01 (16.0...SIVAKAMI\sivag

Step 2: Executed Queries with Explanation

1. Average Room Rates of Different Cities



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a SQL query titled "Average Room Rates of Different Cities". The query is as follows:

```
-- Average Room Rates of Different Cities
SELECT h.city,
       MIN(b.amount) AS min_amount,
       MAX(b.amount) AS max_amount,
       AVG(b.amount) AS avg_amount
FROM   bookings b
JOIN   hotels h ON b.hotel_id = h.hotel_id
WHERE  b.status = 'Stayed'
GROUP BY h.city;
```

The bottom pane shows the results of the query, which is a table with 4 columns: city, min_amount, max_amount, and avg_amount. The results are as follows:

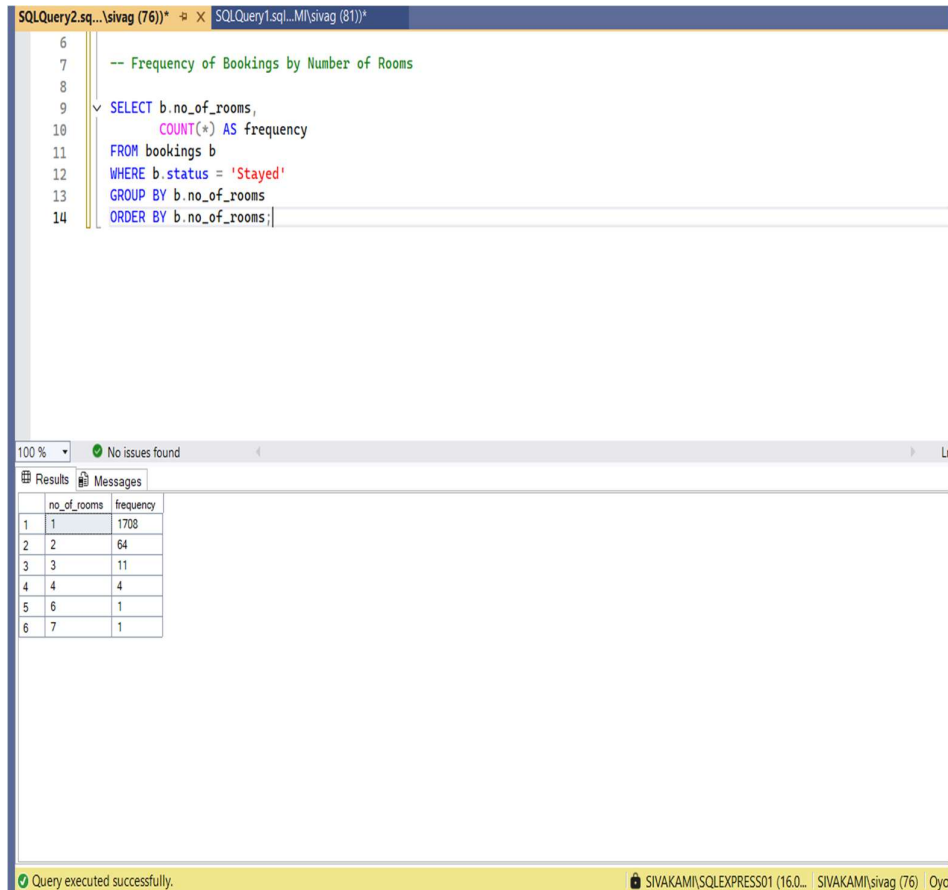
	city	min_amount	max_amount	avg_amount
1	Mumbai	1884	37166	7398.12931034483
2	Gurgaon	926	29375	2735.04718693285
3	Chennai	1429	28882	3677.80303030303
4	Hyderabad	1386	25732	4406.05555555556
5	Bangalore	1104	29692	4079.26997245179
6	Noida	1386	11887	2807.02654867257
7	Delhi	1300	45360	4268.77744807122
8	Jaipur	1039	16232	3543.22535211268
9	Pune	1884	22495	4916.76744186046
10	Kolkata	1685	9812	3779.92857142857

The status bar at the bottom indicates "Query executed successfully." and "SIVAKAMI\SQLEXPRESS01 (16.0.0.0) | SIVAKAMI\sivag (76) | OyoBusine".

Explanation:

This query calculates the minimum, maximum, and average room booking amount for each city by joining the bookings and hotels tables. It filters the data only for customers who actually stayed (i.e., not cancelled or no-show bookings).

2. Frequency of Bookings by Number of Rooms



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a SQL query in the 'SQLQuery2.sql' file. The query is as follows:

```
-- Frequency of Bookings by Number of Rooms
SELECT b.no_of_rooms,
       COUNT(*) AS frequency
FROM bookings b
WHERE b.status = 'Stayed'
GROUP BY b.no_of_rooms
ORDER BY b.no_of_rooms;
```

The bottom pane shows the 'Results' tab with a table containing the following data:

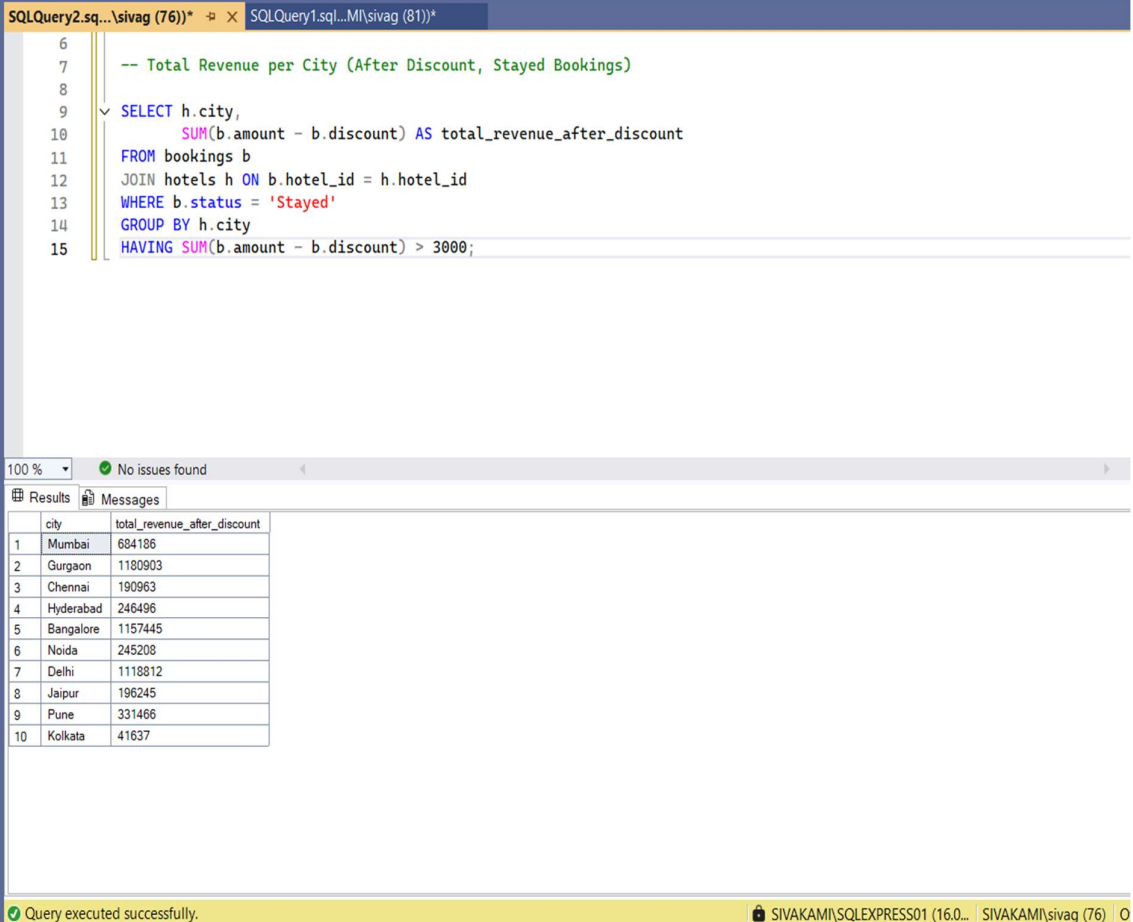
	no_of_rooms	frequency
1	1	1708
2	2	64
3	3	11
4	4	4
5	6	1
6	7	1

The status bar at the bottom indicates 'Query executed successfully.' and the server name is 'SIVAKAMI\SQLEXPRESS01 (16.0.0.1) SIVAKAMI\sivag (76) Oyo'.

Explanation:

This query shows how many times different room quantities were booked (like 1 room, 2 rooms, etc.). It filters only the bookings where the guest actually **stayed** in the hotel.

3. Total Revenue per City (After Discount, Stayed Bookings)



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a SQL query in the 'SQLQuery2.sql' file. The query is as follows:

```
-- Total Revenue per City (After Discount, Stayed Bookings)
SELECT h.city,
       SUM(b.amount - b.discount) AS total_revenue_after_discount
FROM   bookings b
JOIN   hotels h ON b.hotel_id = h.hotel_id
WHERE  b.status = 'Stayed'
GROUP BY h.city
HAVING SUM(b.amount - b.discount) > 3000;
```

The bottom pane shows the 'Results' tab with a table containing 10 rows of data. The table has two columns: 'city' and 'total_revenue_after_discount'.

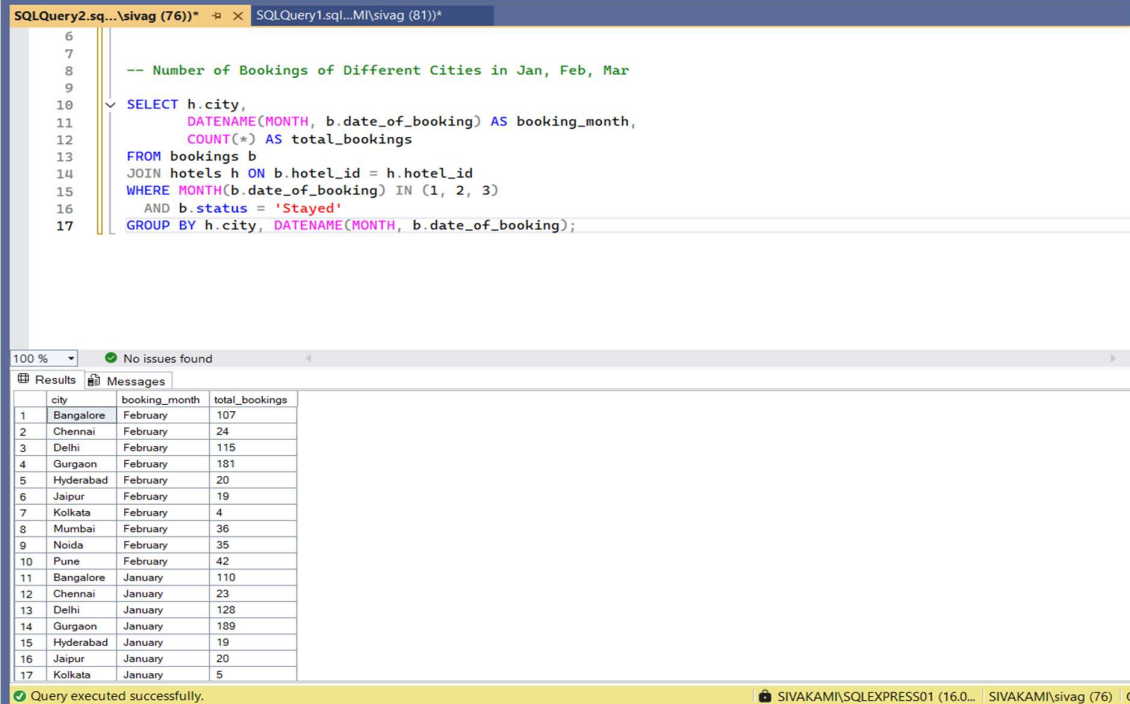
	city	total_revenue_after_discount
1	Mumbai	684186
2	Gurgaon	1180903
3	Chennai	190963
4	Hyderabad	246496
5	Bangalore	1157445
6	Noida	245208
7	Delhi	1118812
8	Jaipur	196245
9	Pune	331466
10	Kolkata	41637

The status bar at the bottom indicates 'Query executed successfully.' and shows the connection path: 'SIVAKAMI\SQLEXPRESS01 (16.0.0) | SIVAKAMI\sivag (76)'.

Explanation:

This query calculates the net revenue (amount after subtracting discount) for each city, considering only stayed bookings. It also filters cities where total revenue is more than 3000.

4. Number of Bookings of Different Cities in Jan, Feb, Mar



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a SQL query in the 'SQLQuery1.sql' file. The query is designed to count the number of bookings for each city in January, February, and March, specifically for bookings where the status is 'Stayed'. The query uses a JOIN between the 'bookings' and 'hotels' tables, filters by the month and status, and groups the results by city and booking month.

```
-- Number of Bookings of Different Cities in Jan, Feb, Mar
SELECT h.city,
       DATENAME(MONTH, b.date_of_booking) AS booking_month,
       COUNT(*) AS total_bookings
FROM bookings b
JOIN hotels h ON b.hotel_id = h.hotel_id
WHERE MONTH(b.date_of_booking) IN (1, 2, 3)
  AND b.status = 'Stayed'
GROUP BY h.city, DATENAME(MONTH, b.date_of_booking);
```

The bottom pane shows the 'Results' tab with a table containing 17 rows of data. The columns are 'city', 'booking_month', and 'total_bookings'. The data is grouped by city and month, showing the total number of bookings for each city in January and February.

city	booking_month	total_bookings
Bangalore	February	107
Chennai	February	24
Delhi	February	115
Gurgaon	February	181
Hyderabad	February	20
Jaipur	February	19
Kolkata	February	4
Mumbai	February	36
Noida	February	35
Pune	February	42
Bangalore	January	110
Chennai	January	23
Delhi	January	128
Gurgaon	January	189
Hyderabad	January	19
Jaipur	January	20
Kolkata	January	5

The status bar at the bottom indicates that the query was executed successfully.

Explanation:

This query counts the number of successful bookings (status = 'Stayed') made in the months of January, February, and March for each city.

Conclusion

This project helped analyze hotel booking data using SQL. We learned to join tables, apply filters, and use aggregate functions to find booking trends, revenue, and customer behavior insights.